

ARRAY

The fundamental data types like int, float, char and double are constrained by the fact that a variable of these types can store only one value at any time. However, we need to address large volume of data for many applications. To store and process such a large volume of data C provides a derived data type known as arrays

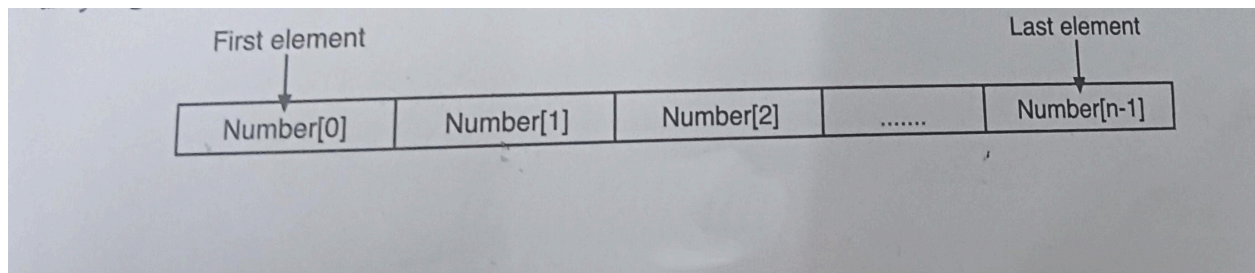
An array is used to store a collection of data, but it is often more useful to think of an array as a collection of variables of the same type. Instead of declaring individual variables, such as number0, number1....number99, we declare one array variable such as numbers and use numbers[0], numbers(1) numbers (99) to represent individual variables. A specific element in an array is accessed by an index (or) subscript

All arrays consist of contiguous memory locations. The lowest address corresponds to the first element and the highest address to the last element.

Types of array

1. One-dimensional Arrays

A list of items can be given one variable name using only one subscript and such a variable is called single-subscripted variable or one-dimensional array. An example of a one dimensional array is given below



The declaration and usage of one-dimensional array is explained below

(a) **Declaring Arrays:** To declare an array in C a programmer specifies the type of the elements and the number of elements required by an array as follows

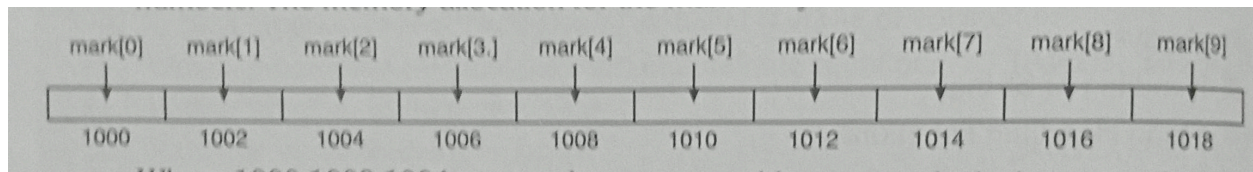
Syntax:

```
data_type arrayname [ arraysize];
```

This is called a single-dimensional array. The arraysize must be an integer constant greater than zero and type can be any valid C data type. For example, to declare 10-element array called mark of type int, use this statement

```
int mark[10];
```

Now mark is an array variable which is sufficient to hold upto 10 integer numbers. The memory allocation for the mark array is



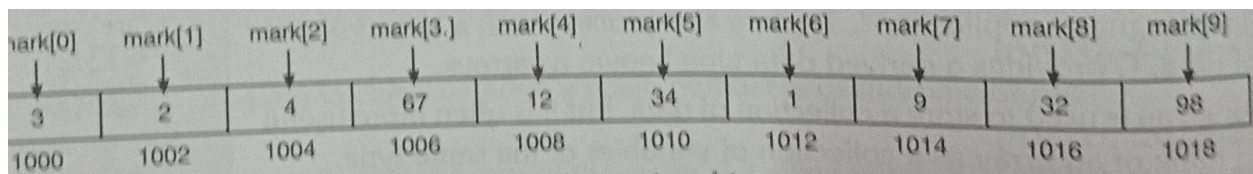
Where 1000,1002,1004 etc are the memory addresses at which the array elements mark[0],mark[1] etc are stored.

(b) **Initialising an array:** We can initialise array in C either one by one or using a single statement as follows

```
int mark[10]={3,2,4,67,12,34,1,9,32,98};
```

The number of values between braces {} cannot be larger than the number of elements that we declare for the array between square brackets [].

For the above array, the memory allocation would be as follows-



If we omit the size of the array, an array just big enough to hold the initialisation is created.

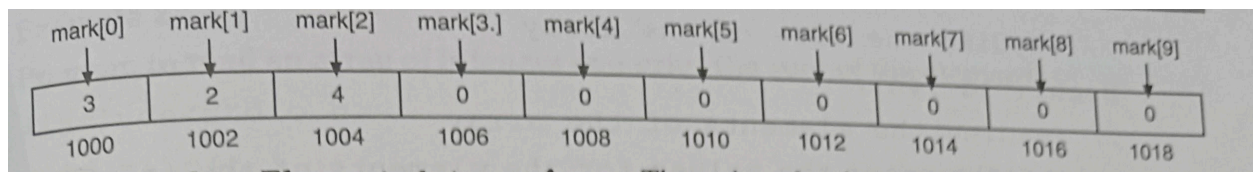
Therefore, if you write

```
int mark[]={3,2,4,67,12,34,1,9,32,98};
```

The array mark size is omitted here. Hence, its size is 10, that is the number of elements initialised to it.

If the number of values in the list is less than the number of elements declared, then only that many elements will be initialised. The remaining elements will be set to 0 automatically. For eg.

```
int mark[10]={3,2,4};
```



(c) Reading Elements into an Array: The values for the elements of an array can be read as follows

```
scanf("%d",&a[0]);  
scanf("%d",&a[1]);  
scanf("%d",&a[2]);
```

If a large number of elements to be read (for ex to read from a[0]....a[99]), then the above way of reading is not advisable. So, to read values into all elements of an array use looping structures like for, while or do while.

Suppose an integer array having 10 elements is declared as given below.

```
int a[10];
```

To read values for this array elements.

```
for (i = 0; i < 10; i++){  
  
scanf("%d",&a[i]);  
}
```

The above statement will read 10 values into the array elements from a[0], a[1]...a[9]

(d) Printing the Elements of an Array

To print values of the elements of an array

```
printf("\n the array elements are...");  
  
for (i=0; i<10; i++)  
{  
printf("\n the element a[%d]: %d ",i,a[i]);  
}
```

The above statement will print the values of the array elements a[0],a[1],.... a[9]

2. Two dimensional array

There is a need to store a table of values often. One-dimensional array is not suitable for such storage.

	Item 1	Item 2	Item 3	Item4
Chennai	23	66	34	544
Delhi	12	23	54	22
Mumbai	5	55	7	668

The above table has three rows with four columns in each row. The table can be considered matrix. We can represent a particular value in a matrix by using two subscripts such as V_{ij} where v refers to the matrix, i refers to the row and j refers to the column.

Syntax:

data_type array_name[row size][column size];

Example : `int a[3][4]` a is a two dimensional array having 3 rows and 4 columns and totally 12 elements.

(a) Memory Representation of 2-D arrays

1. 2-D arrays are stored in contiguous memory location row wise. The address of the first element in the array is base address
2. 3×4 array is shown below in Table

	Column 0	Column 1	Column 2	Column 3
Row 0	<code>a[0][0]</code>	<code>a[0][1]</code>	<code>a[0][2]</code>	<code>a[0][3]</code>
Row 1	<code>a[1][0]</code>	<code>a[1][1]</code>	<code>a[1][2]</code>	<code>a[1][3]</code>
Row 2	<code>a[2][0]</code>	<code>a[2][1]</code>	<code>a[2][2]</code>	<code>a[2][3]</code>

3. Consider 3×4 array is stored in contiguous memory location which starts from 4000. This address is the base address of the array

4 Array element `a[0][0]` will be stored at address 4000, again `a[0][1]` will be stored at the next memory location i.e., elements are stored in row-wise

5. After elements of first row are stored in appropriate memory location, elements of next row get their corresponding memory locations in a continuous flow

6. This is an integer array so each element requires 2 bytes of memory.

b) **Basic Memory Address Calculation:** To calculate the address at which the elements of a two dimensional array is stored, use the following formula

$$a[i][j] = \text{base address} + \text{size of data type} * (\text{number of columns} * i + j)$$

Suppose if $a[0][0]$ is at the address 4000, then the following Table shows address of the other elements of the array

Element	Memory Location
$a[0][0]$	4000
$a[0][1]$	4002
$a[0][2]$	4004
$a[0][3]$	4006
$a[1][0]$	4008
$a[1][1]$	4010
$a[1][2]$	4012
$a[1][3]$	4014
$a[2][0]$	4016
$a[2][1]$	4018
$a[2][2]$	4020
$a[2][3]$	4022

(c) **Initialising 2D arrays:** Two dimensional arrays can be initialised as follows:

```
int mat[3][2] = {1,1,2,2,3,3};
```

The above declaration initialises the first row to one, second row to two and third row to three

```
mat[0][0]=1.      mat[1][1]=2
```

```
mat[0][1]=1.      mat[2][0]=3  
mat[1][0]=2.      mat[2][1]=3
```

The above initialisation can also be done as follows.

```
int mat[3][2] = {{1,1},{2,2},{3,3}};
```

If the values are missing, they are automatically set to zero. For example,

```
Int mat [3][2]={1},{2},{3}};
```

In the above initialisation, the first element of first row is explicitly initialised to 1 and the other elements are initialised to zero automatically. The first element of second row is explicitly initialised to 2 and the other elements are initialised to zero automatically. The first element of third row is explicitly initialised to 3 and the other elements are initialised to zero automatically

To initialise zero to all the elements, one of the following may be used

```
int mat[3][2] ={{0,0},{0,0},{0,0}};
```

```
int mat[3][2]={0,0,0,0,0,0};
```

```
ntmat[3][2] = {{0,0}};
```

(d) **Reading a 2-D array:** An element in 2-dimensional array is read by using the subscripts, ie., row index and column index of the array. The row index ranges from 0 to row_size-1 and the column index ranges from 0 to column size-1. To read a 2D array

```
for(i=0;i<row_size;i++)
```

```
for ( j = 0;j<column_size; j++)
```

```
scanf("%d",&a[i][j]);
```

(e) **Multi Dimensional arrays:** C programming language allows multidimensional arrays. Here is the general form of a multidimensional array declaration

```
data_type name[size1][size2] [sizeN];
```

For example, the following declaration creates a three dimensional integer array

```
int threedim[5][10][4];
```

Two dimensional array is also called as multi dimensional arrays.